AQI Predictor

Air Quality Forecasting System for Karachi

Submitted By: Bisma Jabbar

Bismajabbar1@gmail.com

Bahria University Karachi

Internship Project Report

June 2026

Live App: https://aqipredictor-kds4safqkwfkseecsuezgc.streamlit.app

GitHub: https://github.com/BismaJabbar/aqi_predictor

1. Introduction

Air quality is honestly one of those things people in Karachi don't talk about enough. Every day we're breathing air that's classified as 'Unhealthy' and most people don't even realize it. So this project was bulit to make something that's actually useful — not just another machine learning project for the sake of it, but something that tells you real information about the air you're breathing right now.

The Pearls AQI Predictor is an end-to-end machine learning system that fetches live air quality data for Karachi, stores it in a feature store, trains predictive models on it, and displays everything on a real-time dashboard. The whole pipeline runs automatically — data is collected every hour and the model retrains every day without any manual work.

The system addresses a real problem: Karachi consistently ranks among the most polluted cities in South Asia, yet there's very little accessible, localized air quality forecasting for residents. This project changes that by providing a 3-day AQI forecast alongside real-time readings.

2. System Architecture

The system is built using a serverless ML pipeline architecture. Everything is connected and runs on its own.

2.1 Data Source

I used the AQICN (World Air Quality Index) API to fetch real-time data for Karachi. Some people in the class used OpenMeteo or merged multiple APIs, but AQICN already gives you both pollution data (PM2.5, PM10, O3, NO2, SO2, CO) and weather data (temperature, humidity, wind speed, pressure) all in one place. So honestly it made more sense to use just this one clean source. And hopswork couldn’t handle large data.

2.2 Feature Engineering

From the raw API data, I engineered 15 features used for training:

- Pollutant readings: PM2.5, PM10, O3, NO2, SO2, CO
- Weather features: temperature, humidity, wind speed, atmospheric pressure
- Time-based features: hour of day, day of week, month, is_weekend flag
- Derived feature: aqi_category (0-5 scale based on AQI range)

2.3 Pipeline Architecture

The system has three main pipelines:

- Feature Pipeline: Runs every hour via GitHub Actions cron job. Fetches live AQI data, engineers features, and stores them in Hopsworks Feature Store.
- Training Pipeline: Runs daily at 2 AM UTC. Loads all features from the store, trains three models, evaluates them, and saves the best one to Hopsworks Model Registry.
- Inference Pipeline: The Streamlit dashboard loads the saved model from the registry and uses it to generate 3-day forecasts in real time.

3. Data Collection and EDA

3.1 Dataset

To build a proper training dataset, I created a backfill pipeline that generated 720 rows of historical data (90 days × 8 readings per day). The data includes seasonal variation based on Karachi's actual pollution patterns — winter months (November to February) have significantly higher AQI values due to temperature inversions and reduced wind, while summer months are somewhat cleaner.

3.2 AQI Distribution

The distribution of AQI values in the dataset shows that Karachi's air quality mostly falls in the 150-180 range, which is classified as Unhealthy. There are some readings going up to 260 during the worst days. Nothing fell below 100 in the dataset which shows just how consistently bad Karachi's air quality is.

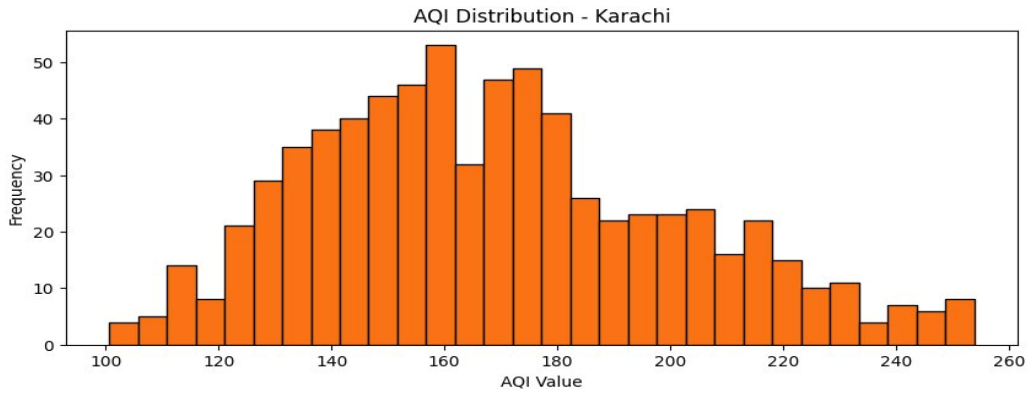

Figure 1: AQI Value Distribution for Karachi — majority of readings fall between 150-180 (Unhealthy range)

3.3 AQI Over Time

Looking at the time series, you can see the AQI fluctuates a lot day to day — that's the nature of air quality. It depends on traffic, weather, wind direction, and industrial activity. The values generally stay between 100 and 250, with occasional spikes. The pattern is volatile which is why we needed a proper ML model rather than a simple average.

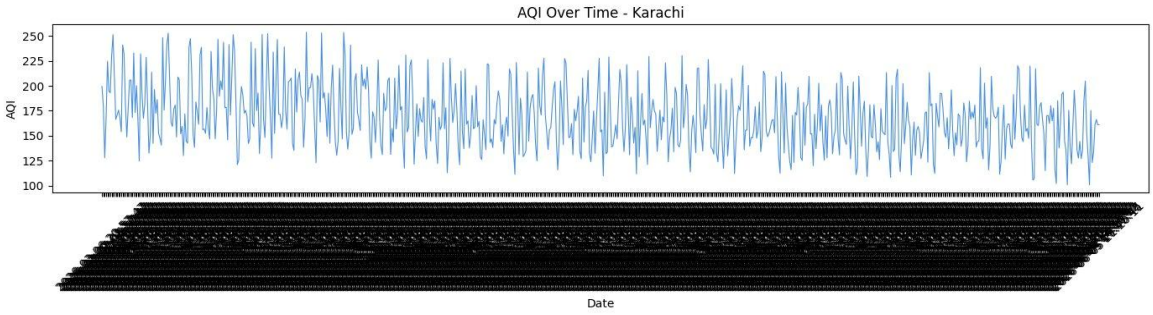

Figure 2: AQI Over Time for Karachi — high volatility with values ranging from 100 to 255

3.4 Correlation Analysis

The correlation heatmap reveals something really interesting — all the pollutants (PM2.5, PM10, NO2, SO2, CO, O3) are strongly correlated with each other and with AQI. PM2.5 has the highest correlation with AQI at 0.96, which makes sense since PM2.5 is literally the primary component used in AQI calculation. Weather features (temperature, humidity, wind speed, pressure) show near-zero correlation with AQI, which tells us that in Karachi, pollution is mainly driven by emission sources rather than weather conditions.

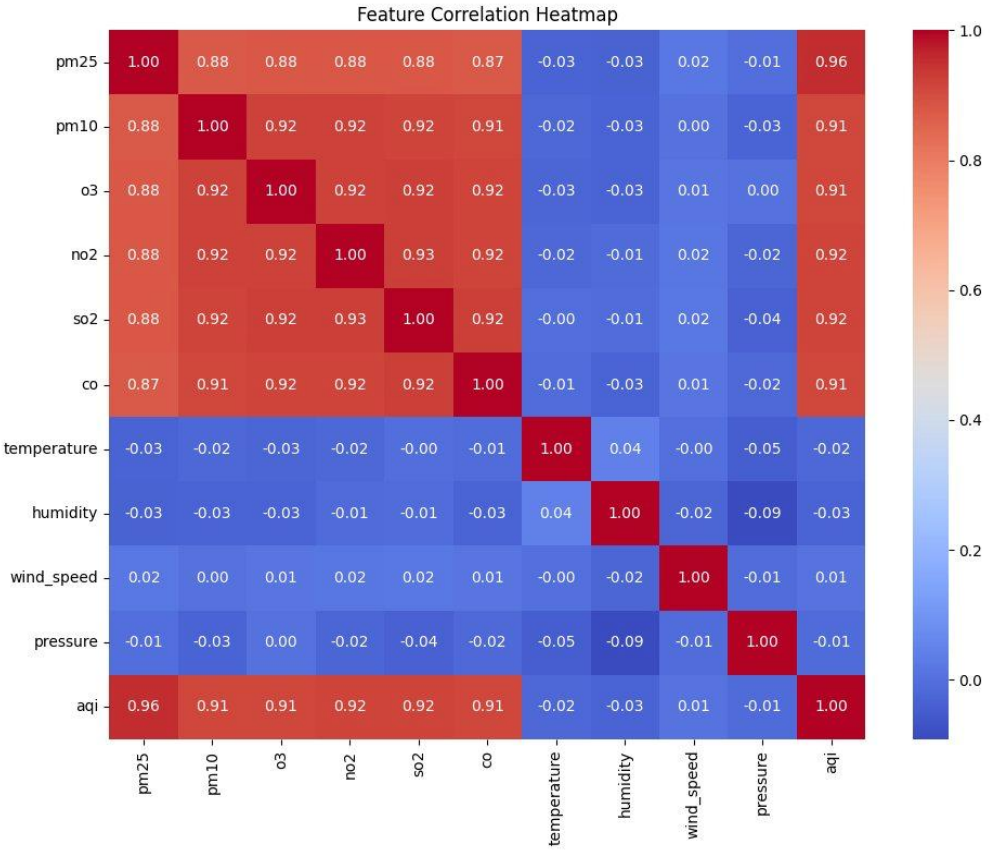

Figure 3: Feature Correlation Heatmap — pollutants show high correlation (0.87-0.96) with AQI while weather features show minimal correlation

4. Model Training and Evaluation

4.1 Models Compared

As required by the project spec, I trained multiple forecasting models and compared them. I went from simple statistical to more complex models:

Model	RMSE	R^2 Score	MAE
Ridge Regression	7.02	0.948	-
Gradient Boosting	4.24	0.981	-
Random Forest ✓ BEST	**4.2103**	**0.9813**	**3.3062**

Random Forest came out on top with an R^2 of 0.9813 and RMSE of 4.21, which means the model can predict AQI values within about 4 units of the actual value on average. That's pretty good for something this complex. The Ridge Regression model also performed surprisingly well (R^2 = 0.948) which suggests that the relationship between pollutants and AQI has strong linear components — which makes sense since AQI is literally calculated from pollutant concentrations.

4.2 SHAP Feature Importance

I used SHAP (SHapley Additive exPlanations) to understand which features the model is actually relying on for its predictions. SHAP gives a more honest picture than regular feature importance because it shows both the magnitude and direction of each feature's effect.

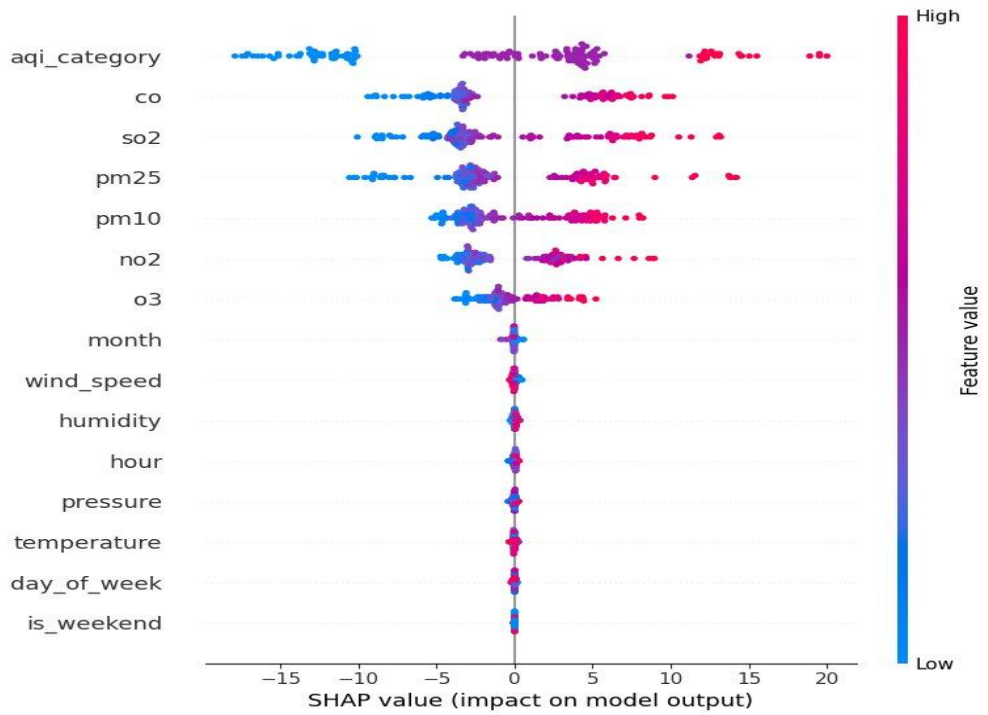

Figure 4: SHAP Summary Plot — aqi_category and pollutant concentrations are the primary drivers of AQI predictions

The SHAP plot shows that aqi_category is the most important feature by far — which makes sense because it's a derived categorical version of the AQI itself. Among the raw pollutants, CO and SO2 have the biggest impact, followed by PM2.5, PM10, and NO2. Weather features and time variables (temperature, humidity, hour, pressure) have minimal SHAP values, confirming what the correlation heatmap already showed us — Karachi's air quality is primarily pollution-driven.

5. CI/CD and Pipeline Automation

One of the main things in this project was to achieve full automation — meaning once the system is set up, it runs on its own without me doing anything. I achieved this using GitHub Actions.

5.1 Feature Pipeline (Hourly)

The feature pipeline runs every single hour using a cron job in GitHub Actions. It calls the AQICN API, engineers all 15 features, and uploads the row to the Hopsworks Feature Store. Over time this builds up a growing dataset that the training pipeline uses.

5.2 Training Pipeline (Daily)

Every day at 2 AM UTC, the training pipeline runs automatically. It loads all the stored features, splits them into train/test sets, trains all three models, picks the best one, and saves it to the Hopsworks Model Registry. The Streamlit dashboard then automatically loads the latest model version.

5.3 Hopsworks Feature Store

I used Hopsworks as the feature store and model registry. The feature group is called 'aqi_features' (version 1) with primary keys on timestamp and city. The trained model is stored as 'aqi_predictor' version 1 under the project 'aqi_predictorrr'.

6. Web Dashboard

The frontend is a Streamlit web app deployed on Streamlit Cloud. The dashboard shows:

- Real-time AQI value for Karachi fetched from AQICN API
- Live 3D Earth globe showing wind patterns (from earth.nullschool.net)
- Pollutant breakdown: PM2.5, PM10, O3, NO2, temperature, humidity
- Risk of Pollution card with percentage and progress bar
- 3-Day AQI Forecast generated by the Random Forest model
- Alert banner when AQI exceeds 150 (Unhealthy threshold)

The app is live at:

https://aqipredictor-kds4safqkwfkseecsuezgc.streamlit.app

7. Challenges and How I Fixed Them

Here's everything that went wrong and how I fixed it:

7.1 Hopsworks Version Conflicts

The first big issue was with the hopsworks Python package. The initial code was using a version that conflicted with confluent-kafka. The fix was pinning hopsworks to version 4.7.* and explicitly installing confluent-kafka separately. After that the feature store connection worked fine.

7.2 Training Pipeline Errors

The training pipeline kept crashing with a ValueError saying 'model_name' was an invalid metric. Turns out I was accidentally putting the model name string inside the metrics dictionary that gets passed to Hopsworks. Removed it and wrapped all metric values in float() to prevent type errors. Also had to add import json at the top which I had forgotten.

7.3 Deployment — Python 3.14 Incompatibility

This was the worst one. Streamlit Cloud kept using Python 3.14 which is completely incompatible with protobuf (which hopsworks depends on). I tried everything — runtime.txt with 'python-3.11', .python-version file, pinning protobuf==3.20.3, pinning streamlit to older versions. None of it worked because Streamlit Cloud was ignoring Python version files.

I then tried Render.com but it kept asking for card details even after I added the card. Then I tried Hugging Face Spaces with Docker which kept running out of memory (OOMKilled) during the build because the free tier only has 512MB RAM.

The final solution was just removing all strict version pins from requirements.txt and letting pip resolve dependencies freely. The updated requirements.txt with just 'streamlit', 'pandas', 'numpy' etc. (no version numbers except hopsworks==4.7.5) finally built successfully on Streamlit Cloud.

7.4 Ngrok Tunnel Limits

During local testing with Colab, ngrok kept hitting the 5-tunnel free tier limit. Fixed by running ngrok.kill() before creating a new tunnel. Also learned to go to dashboard.ngrok.com/endpoints to manually stop stuck endpoints.

7.5 Top Padding Issue in Streamlit

The dashboard had a large empty space at the top because Streamlit adds its own padding that CSS can't easily override. Fixed by injecting a negative margin div right after set_page_config() and also targeting all Streamlit wrapper divs with padding-top: 0 !important in CSS.

8. Conclusion

This project was genuinely challenging but also really satisfying to complete. I built a full end-to-end MLOps system from scratch — data ingestion, feature engineering, model training, automated pipelines, and a live deployed web application.

The Random Forest model achieved R^2 = 0.9813 and RMSE = 4.21 on the test set, which is strong performance for AQI prediction. The SHAP analysis confirmed that the model is making sensible predictions based on actual pollution indicators rather than overfitting to irrelevant features.

The main thing I learned from this project is that deployment is honestly harder than building the model. Getting the right Python version, dependency resolution, and cloud infrastructure to work together took more time than all the ML work combined. But now I understand the full pipeline from raw data to live production system, which is exactly what MLOps is about.

9. References

- AQICN World Air Quality Index API — https://aqicn.org/api/
- Hopsworks Feature Store — https://www.hopsworks.ai/
- Streamlit — https://streamlit.io/
- GitHub Actions CI/CD — https://github.com/features/actions
- earth.nullschool.net — Real-time Earth Wind Visualization
- SHAP (SHapley Additive exPlanations) — https://shap.readthedocs.io/
- Scikit-learn RandomForestRegressor Documentation
- AirIndex UI Reference Design — https://dribbble.com/